

ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería en Sistemas — Ingeniería de Software

Asignatura: Usabilidad y Accesibilidad

SaludEC

Arquitectura Técnica y Análisis de Brechas

Portal de Servicios Públicos de Salud del Ecuador

Autores: Erick Costa, Jonathan Tipán, Javier Quilumba

Repositorio: github.com/zeuspyEC/SaludEC

Despliegue: vitaprevent-b2e34.web.app

Fecha: Junio 2026

Índice

1. Resumen del documento	2
2. Arquitectura general del sistema	2
2.1. Stack tecnológico	2
2.2. Patrón de arquitectura	2
2.3. Estructura de directorios	3
2.4. Modelo de datos Firestore	3
3. Páginas y rutas implementadas	4
4. Componentes implementados	4
4.1. Componentes de layout	5
4.2. Componentes comunes reutilizables	5
4.3. Componentes UI (átomos)	5
5. Análisis de brechas — Requisitos mínimos del sitio	6
5.1. 6.1 Estructura y navegación	6
5.2. 6.2 Contenido	7
5.3. 6.3 Formularios	7
5.4. 6.4 Interactividad	8
5.5. 6.5 Accesibilidad visual	8
6. Brechas pendientes y plan de acción	9
7. Cumplimiento de requisitos del enunciado	10
8. Decisiones de arquitectura relevantes para accesibilidad	10
8.1. Componentes controlados vs no controlados	10
8.2. HTML semántico primero	10
8.3. Tokens CSS centralizados	10
8.4. Gestión de foco en SPA	10
8.5. Firebase Storage no utilizado	11

1 Resumen del documento

Este documento describe la arquitectura técnica completa de SaludEC y realiza un análisis exhaustivo de las brechas entre el estado actual del sistema y los requisitos del proyecto académico (Asignatura Usabilidad y Accesibilidad, EPN). Para cada brecha identificada se indica el estado, la prioridad de resolución y la acción correctiva.

2 Arquitectura general del sistema

2.1 Stack tecnológico

Cuadro 1: Stack tecnológico de SaludEC

Capa	Tecnología	Versión / Notas
Framework UI	React	19.0.0 — hooks, React Router v6
Bundler	Vite	6.3.5 — build, HMR, code splitting
Base de datos	Firebase Firestore	v10 (modular SDK)
Autenticación	Firebase Auth	v10 — email/password
Hosting	Firebase Hosting	CDN global, HTTPS automático
CI/CD	GitHub Actions	Deploy automático a push en main
Estilos	CSS personalizado	Variables CSS (tokens), sin Bootstrap/Material
Linting	ESLint + jsx-a11y	Plugin de accesibilidad en build

2.2 Patrón de arquitectura

SaludEC sigue un patrón **SPA (Single Page Application)** con las siguientes capas:

1. **Enrutamiento:** React Router v6 con `createBrowserRouter` y lazy loading por módulo.
2. **Capa de presentación:** Componentes React organizados en: `ui/` (átomos), `layout/` (marcos de página), `common/` (moléculas reutilizables), `pages/` (organismos completos).
3. **Capa de servicios:** `src/services/` — funciones puras que encapsulan las llamadas a Firestore.
4. **Capa de datos:** Firebase Firestore como base de datos documental NoSQL.
5. **Contextos:** `AuthContext` (sesión del administrador), `ToastContext` (notificaciones globales).

2.3 Estructura de directorios

Listing 1: Árbol de directorios de `src/`

```

1 src/
2   assets/           # Imágenes, íconos SVG, fuentes
3   components/
4     ui/             # Button, Badge, Spinner, Modal
5     layout/         # NavBar, Footer, SkipLink, Breadcrumb,
                        PageWrapper
6     common/         # ArticleCard, ArticleDetail, FormField,
                        Accordion, SectionHero
7   contexts/         # AuthContext, ToastContext
8   hooks/            # useFocusTrap, useAnnouncer
9   pages/            # Home, Nutricion, ActividadFisica, SaludMental
10                      ,
11                      # Prevencion, Biblioteca, Noticias, Contacto,
                        Nosotros,
12                      # Admin
13   services/         # articulos.service.js, recursos.service.js,
                        noticias.service.js
14   styles/           # tokens.css, reset.css, global.css, animations
                        .css
15   utils/            # validators.js, formatters.js
16   config/           # firebase.js, routes.js, constants.js
17   App.jsx           # Árbol de rutas principal
18   main.jsx          # Punto de entrada React

```

2.4 Modelo de datos Firestore

Cuadro 2: Colecciones Firestore y sus campos principales

Colección	Campos principales
articulos	titulo, slug, resumen, contenido (HTML desde editor WYSIWYG o Markdown), modulo, categoria, etiquetas, imagen.url, imagen.alt, autor, creadoEn, actualizadoEn, publicado
recursos	titulo, tipo, url, descripcion, categoria, etiquetas, thumbnail.url, thumbnail.alt, estado
noticias	titulo, slug, resumen, contenido (HTML/Markdown), imagen.url, imagen.alt, creadoEn, categoria, publicado, fuenteUrl
mensajes	nombre, email, asunto, mensaje, fecha, leído

3 Páginas y rutas implementadas

Cuadro 3: Mapa completo de rutas — SaludEC

Ruta	Módulo	Contenido
/	Inicio	Hero 3D, cifras de salud Ecuador, módulos, noticias recientes
/atencion-primaria	Atención Primaria	Artículos MSP/IESS, calculadora IMC, FAQs, filtros
/atencion-primaria/:slug	Artículo	Renderizado HTML o Markdown según formato del contenido almacenado
/vacunacion	Vacunación	PAI Ecuador, beneficios, artículos, FAQs
/vacunacion/:slug	Artículo	Detalle de artículo de vacunación
/salud-mental	Salud Mental	Señales de alerta, artículos, recursos, FAQs
/salud-mental/:slug	Artículo	Detalle de artículo de salud mental
/emergencias	Emergencias	ECU 911, SAMU, hospitales, primeros auxilios, FAQs
/emergencias/:slug	Artículo	Detalle de artículo de emergencias
/biblioteca	Biblioteca	Recursos digitales (PDF, video, enlace) filtrados
/noticias	Noticias	Feed paginado de noticias de salud
/noticias/:slug	Noticia	Artículo de noticia completo
/contacto	Contacto	Formulario accesible con validación
/nosotros	Nosotros	Equipo, misión, valores
/admin	Panel Admin	Panel protegido con Firebase Auth (privado)
/admin/login	Login Admin	Formulario de autenticación

4 Componentes implementados

4.1 Componentes de layout

Cuadro 4: Componentes de layout y su relevancia WCAG

Componente	Función	WCAG implementado
SkipLink	Primer elemento del DOM	2.4.1 Evitar bloques
Navbar	Menú principal + hamburger	2.1.1, 2.4.3, 4.1.2 (aria-expanded, aria-current)
PageWrapper	Wrapper de <main>, title, foco SPA	2.4.2, 2.4.3 (isFirstRender fix)
Breadcrumb	Ruta de navegación	2.4.8, aria-current="page"
Footer	Pie de página con contacto	3.2.6 ayuda consistente

4.2 Componentes comunes reutilizables

Cuadro 5: Componentes comunes y patrones WAI-ARIA

Componente	Función	Patrón ARIA
Accordion	FAQs interactivas	aria-expanded, aria-controls, trigger = <button>
ArticleCard	Tarjeta de artículo	Enlace descriptivo, imagen con alt
ArticleDetail	Renderizado HTML/Markdown adaptivo	Detecta formato del contenido: HTML con dangerouslySetInnerHTML (admin WYSIWYG), Markdown con ReactMarkdown — preserva jerarquía semántica
FormField	Campo de formulario accesible	aria-describedby, aria-invalid, role="alert"
Modal	Diálogo emergente	Focus trap, role="dialog", aria-modal
SectionHero	Encabezado de sección	Semántica H1, aria-labelledby
Spinner	Indicador de carga	role="status", aria-label descriptivo

4.3 Componentes UI (átomos)

- **Button:** Variantes (primary, secondary, ghost, danger). Siempre <button> o <a> nativo.
- **Badge:** Etiqueta de categoría. Solo visual, aria-hidden cuando es decorativo.

- **Spinner:** Indicador de carga con `aria-label` y `role="status"`.

5 Análisis de brechas — Requisitos mínimos del sitio

Esta sección compara los requisitos mínimos del enunciado (Sección 6) con el estado actual del sitio.

5.1 6.1 Estructura y navegación

Cuadro 6: Brecha: Estructura y navegación

Requisito	Estado	Implementación
Encabezado principal	✓	<code><header role="banner"></code> en Navbar
Menú de navegación accesible	✓	Navbar con <code>aria-label</code> , hamburger con <code>aria-expanded</code>
Contenido principal identificado	✓	<code><main id="main-content" tabindex="-1"></code>
Pie de página	✓	<code><footer role="contentinfo"></code>
Jerarquía correcta de títulos	✓	H1 (PageTitle) → H2 (secciones) → H3 (tarjetas)
Enlace para saltar contenido	✓	SkipLink como primer elemento del DOM
Navegación coherente entre páginas	✓	Navbar idéntica, <code>aria-current="page"</code> dinámico
Diseño responsive	✓	Breakpoints 320px / 768px / 1024px

5.2 6.2 Contenido

Cuadro 7: Brecha: Contenido

Requisito	Estado	Implementación
Textos claros y organizados	✓	Contenido real MSP, IESS, ECU 911 con fuentes oficiales
Imágenes informativas con alt	✓	Campo <code>imagen.alt</code> obligatorio en Firestore
Imágenes decorativas ignoradas	✓	Emojis con <code>aria-hidden="true"</code>
Multimedia con transcripción/alt	△	Biblioteca contiene PDFs con título descriptivo. Video: no implementado aún
Tablas accesibles	✓	Tabla IMC con <code><caption></code> , <code>scope="col"</code>
Lenguaje comprensible	✓	Español claro, términos médicos explicados

5.3 6.3 Formularios

Cuadro 8: Brecha: Formularios

Requisito	Estado	Implementación
Etiquetas visibles asociadas	✓	Componente <code>FormField</code> con <code><label htmlFor></code>
Instrucciones claras	✓	<code>Prop hint</code> en cada campo
Mensajes de error accesibles	✓	<code>role="alert"</code> , <code>aria-describedby</code> al error
Validación no depende del color	✓	Icono + texto + <code>aria-invalid</code>
Agrupación lógica de campos	✓	<code><fieldset></code> + <code><legend></code> en formulario de contacto
Confirmación al enviar	✓	Toast con <code>role="alert"</code> y <code>aria-live="assertive"</code>

5.4 6.4 Interactividad

Cuadro 9: Brecha: Componentes interactivos (mínimo 2 requeridos)

Componente	Estado	Detalles de accesibilidad
Accordion (FAQs)	✓	<code>aria-expanded/controls</code> , operable con teclado, patrón WAI-ARIA completo
Modal	✓	Focus trap, <code>role="dialog"</code> , cierra con Escape, devuelve foco
Filtros de categoría	✓	<code><button aria-pressed></code> , roles explícitos, <code>role="group" aria-label</code>
Cubo 3D Hero	✓	Interactivo con mouse/touch. Gira automáticamente. Alt: módulos en Navbar
Calculadora IMC	✓	Formulario con validación accesible, resultado en <code>aria-live="polite"</code>
Paginación Noticias	✓	Botones con <code>aria-label</code> descriptivo

5.5 6.5 Accesibilidad visual

Cuadro 10: Brecha: Accesibilidad visual

Requisito	Estado	Implementación
Contraste suficiente	✓	Ratio mínimo 7.2:1 (texto principal sobre navy-900)
Fuentes legibles	✓	Mínimo 16px body, escalable con <code>rem</code>
Espaciado adecuado	✓	Tokens CSS: <code>-space-*</code> con valores mínimos WCAG
Foco visible	✓	<code>:focus-visible</code> outline 3px #1e88e5
Zoom sin pérdida	✓	Responsive hasta 200 % sin scroll horizontal
No depender solo del color	✓	Errores: icono + texto + color. Categorías: texto + color
Reducción de movimiento	✓	<code>@media (prefers-reduced-motion)</code> implementado en CSS

6 Brechas pendientes y plan de acción

Brechas identificadas — Prioridad alta

Las siguientes brechas deben resolverse antes de la entrega final:

1. **WCAG 2.5.7 (AA) — Movimientos de arrastre:** El cubo 3D usa arrastre. La alternativa (rotación automática + acceso por Navbar) existe pero no está explícitamente documentada en el código con `aria-label` que lo indique. *Acción:* Añadir `aria-label="Cubo decorativo. Usa el menú de navegación para acceder a los módulos"` al contenedor del cubo.
2. **WCAG 3.2.6 (A, WCAG 2.2) — Ayuda consistente:** Falta enlace de ayuda contextual en cada módulo. *Acción:* Añadir sección de contacto/ayuda visible en cada página de módulo, o enlace permanente al formulario de contacto en el contenido principal.
3. **Multimedia con transcripción:** El sitio no tiene videos. La Biblioteca tiene PDFs enlazados con títulos descriptivos, pero no transcripciones de audio. *Acción:* Para la entrega final, documentar explícitamente que los recursos PDF incluyen texto alternativo en su título y descripción.

Brechas de menor prioridad

- **i18n (idiomas indígenas):** No implementado. Fuera del alcance actual.
- **PWA offline:** Service Worker no configurado. No es requisito del enunciado.
- **Directorio de establecimientos MSP:** Requiere integración de API externa. Trabajo futuro.

Brecha resuelta en Sprint 2 (al 02/07/2026)

- **Panel Admin completo:** ✓ Implementado. CRUD completo de artículos, noticias, recursos y mensajes. Incluye servicio de limpieza (`cleanup.service.js`) que asigna imágenes verificadas y únicas por sección a todos los documentos de Firestore automáticamente.
- **Imágenes únicas por sección:** ✓ Implementado. Cada artículo recibe una fotografía visualmente distinta dentro de su módulo; sub-routing diferencia artículos del mismo tema con títulos distintos.
- **Renderizado HTML semántico:** ✓ Implementado. `ArticleDetail` y `NoticiasDetalle` detectan el formato del contenido (HTML vs Markdown) y lo renderizan preservando la jerarquía semántica.

7 Cumplimiento de requisitos del enunciado

Cuadro 11: Estado global de cumplimiento — Sección 6 del enunciado

Área	Cumplimiento	Observación
6.1 Estructura	100 %	Todos los requisitos implementados
6.2 Contenido	85 %	Pendiente: multimedia con transcripción
6.3 Formularios	100 %	Formulario contacto + calculadora IMC
6.4 Interactividad	100 %	6 componentes interactivos accesibles
6.5 Accesibilidad visual	100 %	Todos los requisitos implementados
7.1 WCAG 2.2	96 %	2 criterios parciales (2.5.7, 3.2.6); 5 correcciones adicionales aplicadas en Sprint 2
7.2 WAI-ARIA	100 %	Uso correcto y selectivo
7.3 ATAG	100 %	Documentado en informe
7.4 UAAG	100 %	Probado en 5 navegadores + NVDA

8 Decisiones de arquitectura relevantes para accesibilidad

8.1 Componentes controlados vs no controlados

Se usan únicamente componentes controlados en formularios (`value + onChange`), lo que garantiza que el estado del formulario esté siempre sincronizado con React y que los mensajes de error (`aria-describedby`) apunten a contenido actualizado.

8.2 HTML semántico primero

La regla de desarrollo es: si existe un elemento HTML nativo para la función, se usa ese. No se construyen botones con `<div onClick>`. Esta decisión elimina de raíz las violaciones más comunes de WCAG 4.1.2.

8.3 Tokens CSS centralizados

Los colores, tipografías y espaciados se gestionan como variables CSS en `tokens.css`. Esto garantiza que cualquier cambio de color sea retroactivo a todos los componentes, y que el ratio de contraste se mantenga sin verificación individual en cada componente.

8.4 Gestión de foco en SPA

El problema clásico de las SPA es que al cambiar de ruta, el DOM se actualiza pero el navegador no mueve el foco automáticamente, dejando al usuario de lector de pantalla

sin anuncio del cambio de contexto. La solución implementada en **PageWrapper**:

- En **carga inicial**: no mover el foco (Tab va: SkipLink → Navbar — corrección WCAG 2.4.3).
- En **navegación SPA**: mover foco al `<main tabindex="-1">` para que el lector anuncie el nuevo título de página.

8.5 Firebase Storage no utilizado

El plan gratuito de Firebase tiene límites en Storage. Las imágenes se referencian como URLs externas (CDN de terceros) o se almacena solo la URL en Firestore. Esto evita costos y mantiene el proyecto en el plan Spark (gratuito).